Design of a FPGA Based Data Acquisition System for Radio Astronomy Applications

Yogindra Abhyankar[1], Sajish C[1], Pallavi Kulkarni[1], C.R.Subrahmanya[2]

[1]Centre for Development of Advanced Computing, University of Pune campus, Ganeshkhind, Pune 411 007, India.

[2]Raman Research Institute, Bangalore 560 080, India

Email: crs@rri.res.in; yogindra@cdacindia.com

Abstract— **In this paper, we describe a high-speed, PCI based data acquisition system using a Xilinx FPGA for Very Long Baseline Interferometry. This system accepts data through LVDS interface and has time keeping options through external clocks for providing an accurate time tagging on the acquired data. It has provision for on-line flagging on selective data samples. Using the host interface, the system can communicate for data transfer through DMA. User can allocate multiple buffers in the host memory where the acquired data can be transferred. The system is very useful for a variety of high-speed signal processing applications in Radio astronomy and other areas.**

Index Terms—**Data Acquisition, Digital Signal Processing, Radio Astronomy, VLBI, FPGA, Data Processing**

I. Introduction

Conventionally, the role of a high speed data acquisition systems is to accept data through an input connector and transfer the input byte stream to the host memory using an appropriate DMA mechanism. In many applications in radio astronomy and signal processing, there are often additional requirements like accurate time tagging, support for bursts, externally controlled data validity etc., which have generally been addressed by building custom systems tailored for a specific requirement. For instance, in the case of Very Long Baseline Interferometry (VLBI), signals received by widely separated radio telescopes are recorded and combined off-line to form equivalents of interferometers with baselines of thousands of kilometers. In such an experiment, the desired information is contained in the temporal behavior of the correlation between the signals arriving at different stations. In order to meaningfully estimate the correlation, it is necessary to compensate for relative propagation delays to accuracies of tens of nanoseconds or better, and also synchronize the sampling clocks and other oscillators used for frequency translation at each station locked to each other to very high degree of relative phase stability [1]. A variety of expensive special purpose hardware has been built to meet this requirement, which have seldom been used in other contexts. The availability of Global Positioning System (GPS) simplifies time synchronization, provided time tagging using a GPS or GPS disciplined oscillator is built into the data acquisition system.

In view of the availability of reconfigurable modules like the Field Programmable Gate Arrays (FPGA), it is now possible to eliminate the need for such special purpose hardware by providing a common hardware and host interface with a provision for a variety of application specific features realizable by simply reprogramming the FPGA on the data acquisition card. By providing such a feature as an add-on option to a commercially available PC, the data acquisition can be combined with significant real time processing and data recording. The FPGA based data acquisition systems have been used in many applications, including nuclear research [2] and astronomy [3].

This paper presents a powerful and programmable FPGA based data acquisition system using Xilinx XCV300 device. First, a brief description of the hardware features is given followed by an outline of the functional blocks implemented in the FPGA for a general purpose data acquisition system with a PCI interface to the host. All our tests were conducted using a Pentium PC running under Linux operating system as the host.

II. FPGA Based Data Acquisition System

Our design is based on a Xilinx virtex [4] FPGA, used for host interface via PCI bus and for performing a variety of other operations, that we feel, are important for a data acquisition system. Figure 1 shows a block diagram of the system. It operates on a 32-bit, 33 MHz PCI bus. It is a high speed, PCI based system that accepts 8-bit differential input through Low Voltage Differential Signaling (LVDS) parallel interface or

0-7803-8656-6/04/$20.00 ©2004 IEEE

through a high-speed 622 Mbps serial LVDS interface. Besides this interface with the external world, it can communicate with the host for data transfer using DMA. There is an on-board support for 128MB of SDRAM. The SDRAM is useful for storing the input, intermediate as well as the final results. It also has a provision for a daughter card mating with a micro strip connector, allowing additional resources to the system.

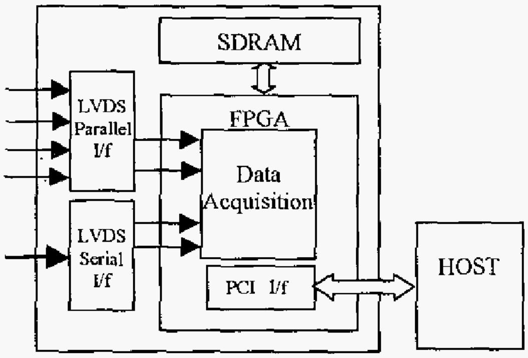

Fig. 1. Data Acquisition System Block Diagram

The data samples acquired by the card are stored into the FIFOs within the FPGA. User can allocate multiple buffers in the system memory where the acquired data can be transferred. There are two modes of operations, mainly based on the input data format. After the data acquisition and time stamping, the data may be passed to the user, based on the selected options.

The data acquisition system operates using the three basic steps: (1) Reset the system (2) Set the acquisition arguments and (3) Start/ Stop the acquisition. All of these operations are indicated to the card using slave write transactions by the host.

The card acquires data through a LVDS interface. This allows for an 8-bit data input, data valid, external reset, reference clock for the time stamping logic and a clock for the entire design. The data transfer from the LVDS interface is controlled by the host, using START and STOP commands. The input data is sampled on the strobe and the data valid status.

The sampled data is collected into an internal FIFO on the FPGA. When the FIFO is almost full, the card initiates bus mastering and dumps the data into the host memory.

The timestamp counters can be driven by an external clock or by the data strobe. In critical applications, it is necessary to use a GPS disciplined oscillator for the reference clock. It can be reset, by connecting the reset pin to a one-sec pulse provided by the GPS.

The host memory consists of buffers, each of size 128 KB. The number of such buffers is user defined. Data is transferred to these buffers sequentially. After the last buffer is full, the next transfer is done to the next cyclic buffer.

Data acquisition is stopped when the FIFO is full or a STOP command from the host is received or the invalid data is indicated by the data valid signal.

The DMA to the host is discontinued when one buffer full of data is transferred to the host or invalid data is detected.

For the last valid data of every buffer, the timestamp counter value is recorded in the timestamp register.

The status register is continuously updated that records the number of invalid data encountered after an initialization command and the setting of the mode for the current acquisition.

A. Architectural Overview

In this section we describe the architectural blocks, mainly the register sets within the FPGA of the system. Figure 2 shows a block diagram of the register set.

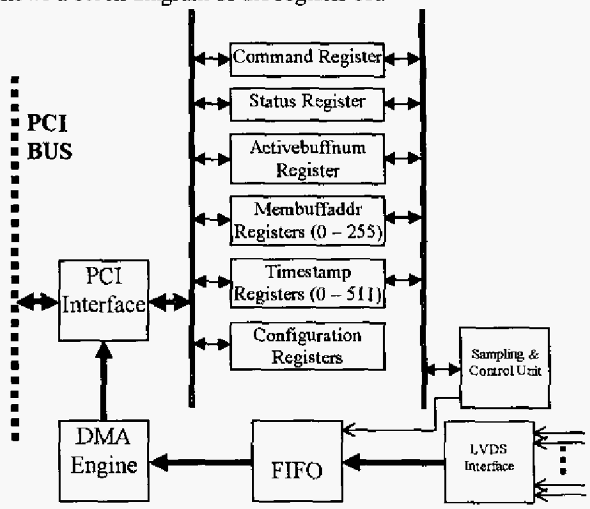

Fig. 2. Register Level Block

There are five important registers used in the design: Command, Timestamp, Status, Activebuffnum and the Membuffaddr.

The Command register is a 32-bit write-only register used for setting different parameters of the acquisition and for initializing the system. It allows for the selection of acquisition mode, reset options, number of host memory buffers, data valid controls, timestamp clock selection and the Start/Stop commands.

The acquisition mode allows for 8-bit data sampling on rising edge or on both edges of the strobe. The reset option allows for an internal or external reset control for resetting the entire data acquisition logic and the timestamp counters. The number of host memory buffers is user selectable with a maximum value of 255. The data valid control enables or disables the data valid signal used for the data sampling. The timestamp control allows for the selection of time stamp clock. The Start/Stop command indicates a start or a stop operation on the LVDS side of the data acquired on the card.

The Time Stamp register is a 64-bit register to store the time stamped value for every packet, packet number and a flag indicating if the data within a packet is complete valid or partial valid.

The Status register is a 32-bit read only register used to keep track of the invalid data count and the mode of current data acquisition. The invalid data count records the number of data faults encountered between Start and Stop. It is reset with the timestamp reset.

The Activebuffnum register indicates the buffer number to which the current data transfer is taking place.

The Membuffaddr register is a set of 256 registers to store the initial address of host memory regions. The host has to pass these addresses that are used later during the DMA operation.

B. *Experimental Setup*

Our PCI based data acquisition system is attached with a host running on Linux. A low level driver performs the PCI configuration and other standard initialization for the system. The high level code written in 'C', gives Start/Stop, Initialization, Reset and other commands to the system.

For bench trials, an experimental setup was created to acquire data stream supplied by a pattern generator in the form of RS-422 differential outputs. A simple circuitry for converting the RS-422 to LVDS signal levels was designed and interfaced to the main FPGA based data acquisition card that accepted data in the form of 8-bit LVDS differential data with strobe. On the current system based on XCV300, maximum achievable PCI bandwidth is 90 MB/s. However, if one uses the on-board SDRAM as a buffer, one can sustain a burst of size up to 128 MB for data entering the input port at speeds up to 600 MB/s.

The card designed by us also underwent field trial for an experiment involving continuous acquisition of data from a satellite for 8 days at 10 MB/s with digital filters implemented through software on the host system. The details of this experiment will be published elsewhere (C.R. Subrahmanya et al.,in preparation). For the present, we would like to note that about 7 terabytes of data were flown into a Pentium host through this system, by sustaining input data rate of 10 MB/s for about 200 hours but still leaving the PC free for performing significant amount of real time computing.

Using such a card on a PC equipped with recently available high-speed tape recorders (e.g., SDLT 600 tape drive), it is possible to sustain continuous data recording at about 40 MB/s for several hours.

III. Conclusions

In this paper, we have described our design of a FPGA based high bandwidth data acquisition system for radio astronomy research. This system is very powerful for precise time stamped data acquisition. The design is flexible enough to apply on other applications that use on-line data recording and time stamping. The upgraded version of this design based on Xilinx XC2V1000 device will increase the PCI bandwidth as well as provide additional logic space.

Acknowledgment

This project is a part of the collaboration between Raman Research Institute (RRI), Bangalore and C-DAC Pune. We thank the Ministry of Information Technology (MIT) and the Department of Space, Government of India for the support. We would like to thank the anonymous referees for making suggestions, which have helped in improving the manuscript.

References

[1] Thompson A. R., Moran, J.M., Swensen G.W., "Interferometry and Synthesis in Radio Astronomy", 2nd edition, Wiley 2001.

[2] Nurdan, T. Conka-Nurdan, H.J. Besch, B. Freisleben, N.A. Pavel and A.H.Walenta, "FPGA-based data acquisition system for a Compton camera," Nuclear Instruments and Methods in Physics Research A 510 (2003), pp. 122-125.

[3] F Morgan, T Bennett, A Shearer and M Redfern, "An FPGA-based Time Resolved Data Acquisition System for Astronomical and Other Applications," Available: http://www.celoxica.com/techlib/files/CEL-W0307171HR6-14.pdf.

[4] Virtex 2.5 V Field Programmable Gate Arrays Data book, Xilinx Inc., April 2001.